

Herramientas de análisis de datos para actuarios

Del Excel al modelo productivo:
reproducible, auditable y defendible.

Agenda

- 1 Quiénes somos: Actuarial + Data
- 2 El problema: ponerle precio al riesgo
- 3 GLM: el modelo como motor técnico
- 4 Del Excel al modelo productivo
- 5 Databricks como plataforma de trabajo
- 6 Demo: del dato crudo al modelo trazable
- 7 Ideas para llevarte

¿Cuánto le cobrarías a cada uno?

Ana

- 25 años
- No fuma
- Peso saludable
- Sin hijos

Carlos

- 55 años
- Fuma
- Sobrepeso
- Dos hijos a cargo

¿Les cobramos lo mismo? Si no, **¿cuánto a cada uno — y cómo lo justificas?**

Guarda tu respuesta. Al final, el modelo va a contestar — y vamos a poder explicar exactamente por qué.

Quiénes somos

El equipo de PwC risk modelling service se compone de 3 sub-equipos.
Todos enfocados en resolver un mismo problema: **convertir datos en decisiones defendibles.**

P&C

Seguros generales: auto, hogar. Tarifas basadas en datos de siniestros.

Vida

Mortalidad, longevidad, productos de largo plazo.

Data

Pipelines, modelos y plataforma. Devs trabajando codo a codo con actuarios.

El criterio es del actuario. La tecnología **amplifica**, no decide.

2

El problema:
ponerle precio al
riesgo

¿Qué hace un actuario de pricing?

Poner precio al riesgo.

Un asegurado transfiere su incertidumbre a la compañía. El actuario decide cuánto cobrar para que la compañía pueda pagar los siniestros, el precio refleje el riesgo real, y el negocio sea sostenible.

Cuatro pasos, del dato a la tarifa

1. Definir el problema

¿Qué queremos predecir? Aquí: el costo esperado de cada asegurado.

2. Estructurar datos

Reunir las características de cada riesgo en una tabla limpia y modelable.

3. Automatizar el modelo

GLM con variables de riesgo en un pipeline reproducible y versionado.

4. Output accionable

Tabla de tarifas con relatividades interpretables...
como la que vamos a construir en vivo.

Dos preguntas distintas — ¿cuántos eventos ocurren? (frecuencia) y ¿cuánto cuesta cada uno? (severidad). Hoy predecimos un **monto**: el costo esperado. El mismo método sirve para la frecuencia, cambiando la familia.

3

GLM: el modelo
como motor
técnico

¿Qué es un modelo?

Una función que aprende de datos históricos para predecir algo nuevo. Le mostrás muchos ejemplos del pasado, descubre patrones, y los aplica a casos que nunca vio.



Dos grandes familias de modelos supervisados

Regresión

la de hoy

Predice un **número** continuo.

Ejemplo: ¿cuánto costará el seguro? \$3.800 + \$52.500 + ...

Clasificación

Predice una **categoría**.

Ejemplo: ¿caduca la póliza? ¿hay fraude? sí / no

Pariente cercano: la regresión logística. En actuarial, también las q_x de las tablas de mortalidad son probabilidades.

Hoy hacemos regresión — predecimos un monto (charges).

El GLM que viene es justamente un modelo de regresión, ajustado a la forma de nuestros datos.

¿Qué es el GLM?

El GLM (Modelo Lineal Generalizado) modela la relación entre variables de riesgo y resultados esperados, transformando datos en decisiones de pricing consistentes, explicables y accionables.

Basicamnte: una fórmula que conecta las características de cada asegurado con su costo esperado — y te dice, en números, cuánto pesa cada característica.



Captura relaciones complejas

Usa múltiples variables a la vez e identifica el impacto individual de cada factor.

Es interpretable

Cada coeficiente tiene significado de negocio y se lo podés explicar a un regulador. **La explicabilidad es oro.**

Los tres ingredientes del GLM

1 Entradas

Las variables explicativas (X): edad, zona, tipo de auto... El modelo las toma para estimar Y.

2 Modelo

El GLM combina X a través de la **función de link** (ej. log-link) para generar una predicción.

3 Salida

La predicción de Y (frecuencia o severidad), válida y consistente con la distribución elegida.



Distribución del modelo



Conecta X con la media de Y



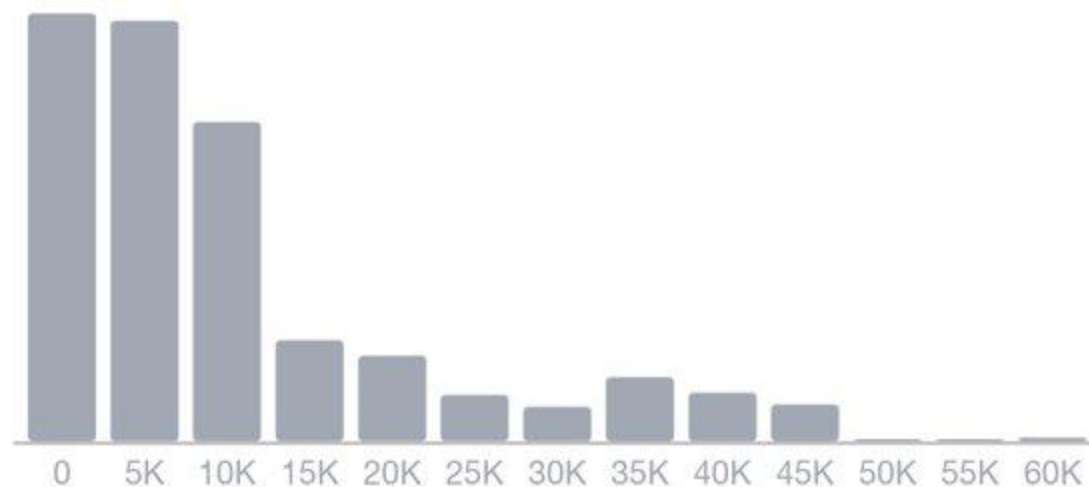
Predicciones válidas

¿Por qué Gamma? El dato lo dice

Antes de elegir el modelo, miramos cómo se distribuye lo que queremos predecir: el charges de 1.338 pólizas reales.

Distribución de charges

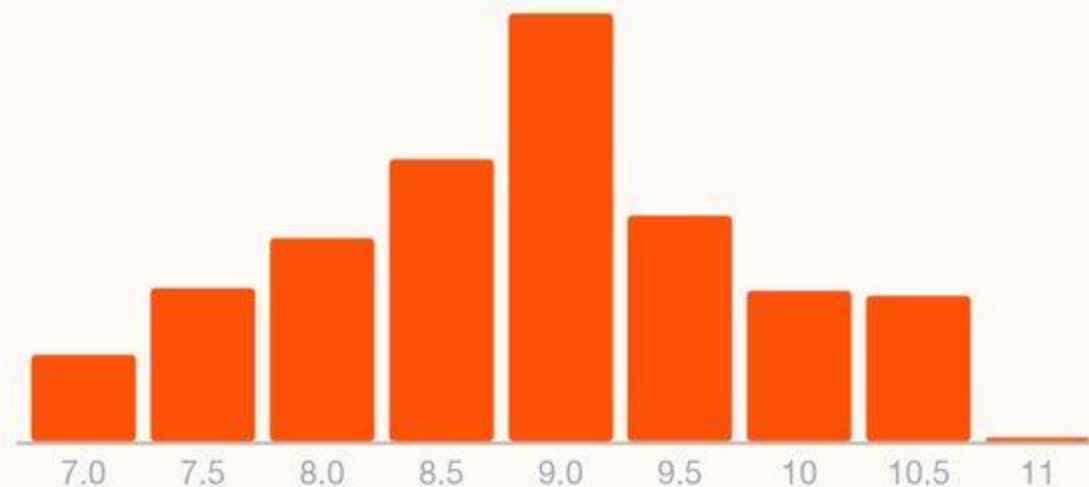
cola larga ..



No es una campana: se amontona abajo y arrastra una **cola hacia la derecha**. Una regresión lineal normal asume simetría → acá no aplica.

Distribución de $\log(\text{charges})$

casí normal ✓



Al tomar **logaritmo**, la cola se acomoda y queda casi simétrica. Esa es la pista: una **Gamma + link log** modela esto naturalmente.

El dato eligió el modelo, no al revés.

Costo positivo + cola larga + log que normaliza = **Gamma con enlace logarítmico**.

El GLM en nuestro caso: 3 decisiones, 1 modelo

1. Distribución (familia)	¿Cómo se comporta lo que quiero predecir?	charges es siempre positivo y con cola larga → Gamma
2. Función de enlace (link)	¿Cómo conecto las variables con el resultado?	Link logarítmico → los efectos son multiplicativos
3. Predictor lineal	¿Qué variables de riesgo uso?	edad, BMI, hijos, sexo, región, fumar

Con link log, **$\exp(\text{coeficiente})$ = relatividad.**

Ejemplo: fumar → **x4.5** la prima. Eso es lo que va a una tabla de tarifas.

Mismo método, distinta familia

Cada combinación familia-link responde a una pregunta distinta del problema.

Gamma + log

la de hoy

Predice un **monto** (severidad): valores positivos continuos, como el costo de hoy.

¿Cuánto cuesta?

Poisson + log

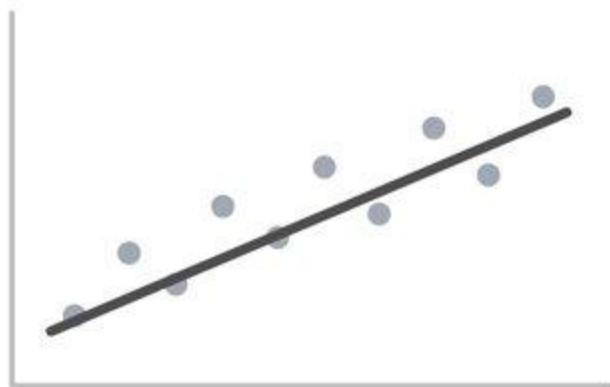
Variable: **frecuencia** de siniestros. Conteos ≥ 0 . El clásico de auto/hogar.

¿Cuántos siniestros habrá?

Hay más combinaciones (prima pura, Tweedle...) — estas dos son las que importan para la charla.

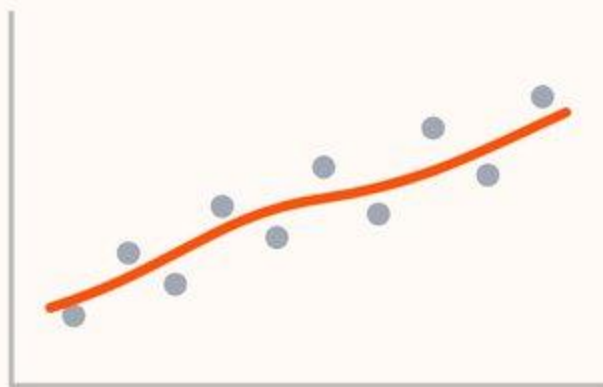
Ni de menos, ni de más: el ajuste justo

Un modelo tiene que aprender el patrón real, no memorizar el ruido. Demasiado simple no captura nada; demasiado complejo se ajusta a casos puntuales y falla con datos nuevos.



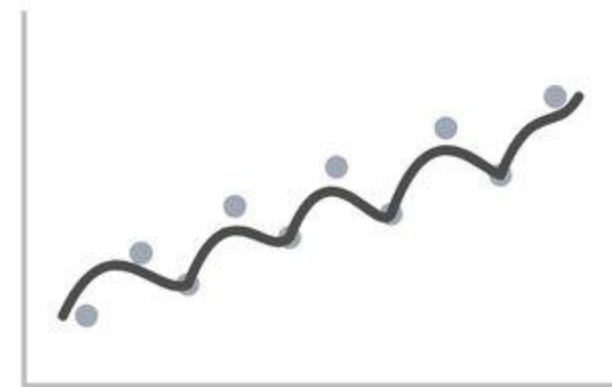
Underfitting

Demasiado simple. Ignora la forma de los datos y se equivoca en todos lados.



Buen ajuste

Captura la tendencia real y generaliza a casos nuevos. Acá apunta el GLM.



Overfitting

Demasiado complejo. Memoriza el ruido de estos datos y falla con otros nuevos.

Por eso entrenamos con una parte de los datos y **probamos con otra** (train/test): para saber si el modelo de verdad generaliza, no si solo memorizó. Lo vas a ver en la demo.

4

Del Excel al
modelo
productivo

Antes y después: pricing actuarial moderno

Antes — Excel manual	Después — Código reproducible
Archivos locales, copiar y pegar	Datos gobernados, ingesta automatizada
Fórmulas manuales	Código versionado
Resultado difícil de repetir	Ejecución reproducible
Modelo como archivo, auditoría manual	Modelo gobernado, evidencia automática

No es “dejar Excel”. Es que tu análisis se pueda **repetir, auditar y defender** — seis meses después, ante un regulador. Eso lo vas a ver en la demo.

¿Qué problema resuelve Databricks?

Tradicional: todo separado

- Datos en bases, archivos o exports
- Código en notebooks locales y scripts sueltos
- Outputs en Excel, modelos sin versionar

Databricks: todo centralizado

- Datos** — tablas gobernadas (Unity Catalog)
- Código** — notebooks y jobs versionados
- Modelos** — registrados (MLflow), con auditoria

Hoy en la demo tocás dos: el **Workspace** (donde corre el notebook) y **Unity Catalog** (datos gobernados). MLflow y Jobs los nombramos al pasar.

Primero mirar el dato, después modelar

Ningún modelo arregla datos malos. Antes de entrenar, hacemos un **chequeo**: ¿está completo? ¿tiene sentido? ¿hay cosas raras?

¿Está completo?

Buscamos valores faltantes. En nuestra tabla: **0 nulos** en 1.338 filas × 7 columnas. Limpio.

¿Tiene sentido?

Chequeamos rangos: edad **18–64**, BMI plausible, categorías esperadas (sexo, región, fumador).

¿Hay cosas raras?

Outliers y forma de la variable a predecir: el costo tiene **cola larga** — pista clave para elegir el modelo.

Este paso, el **data profiling**, decide medio modelo: la cola larga del costo es exactamente lo que nos lleva a una **Gamma**. Lo vemos en vivo en la demo.

Lo que vas a ver ahora

- 1 **Cargar** un CSV crudo (lo que normalmente vive en un Excel)
- 2 **Mirar** el dato: ¿está completo? ¿tiene sentido? (data profiling)
- 3 **Entender** qué mueve el costo (un par de gráficos)
- 4 **Entrenar** el GLM — una línea de código
- 5 **Leer e interpretar** el modelo: relatividades que defendés ante un regulador

Poco SQL, poco Python. El foco está en pensar como actuario, no en programar.

6

DEMO EN VIVO · Notebook en Databricks

Demo time

¿Y esto cómo escala? De la demo a producción

Lo que viste hoy

Un dataset simple (1.338 filas), un GLM Gamma para el costo, y relatividades interpretables. Caso didáctico, método real.

El mismo método en un ramo real

En P&C (auto/hogar) se modela la frecuencia de siniestros con un GLM Poisson sobre millones de pólizas. Cambia la familia, no el método.

Lo que agrega Databricks

Datos gobernados (Unity Catalog), cada entrenamiento reproducible (MLflow), y el modelo operando solo (Jobs).

Data_Ingestion

Train_GLM

Batch_Inference

Promote_Champion

Monitoring

5 ideas para llevarte

- 1 **Programá:** Python y SQL son herramientas actuariales. Aprendelas.
- 2 **Explicá:** un modelo que no se entiende, no se aprueba.
- 3 **Goberná:** versioná, documentá y auditá cada paso.
- 4 **Combiná** estadística con negocio: el riesgo tiene que tener sentido.
- 5 **Pensá en producción:** el modelo opera, no solo entrena.

El actuario que entiende el riesgo, gana

Modelá, explicá y defendé tus decisiones con herramientas reproducibles y auditables.

Te pasamos **todo**: el notebook, el dataset y una guía para crearte una cuenta **gratis** en Databricks Community Edition y reproducir esto en casa.



contact-us-cyan-ten.vercel.app

APÉNDICE - solo si preguntan

Arquitectura de referencia (Lakehouse)

Cómo se ve el stack completo end-to-end. No la mostramos en el flujo — está acá por si alguien técnico pregunta.

Source

Pólizas, siniestros, streaming, externos

Ingest

ETL / workflows, ingesta automatizada

Transform & Store

Delta Lake, tablas gobernadas (Unity Catalog)

Model & Serve

GLM / ML, MLflow, batch & online scoring

Analysis / Output

BI, tarifas, dashboards, monitoreo

Gobierno transversal a todas las capas: permisos, linaje y auditoría. El Lakehouse es a la vez data warehouse y plataforma analítica.